	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering**	
Assignment No : 7		

Study Declarative and Scripted Pipeline in Jenkins and create a simple Declarative Pipeline with Build, Test, and Deploy stages..

Objective

To understand the concept of Jenkins pipelines and implement a Declarative Pipeline using Jenkinsfile with Build, Test, and Deploy stages.

Description

This experiment demonstrates the use of Jenkins Pipeline to automate the software development process. A Declarative Pipeline is created using a Jenkinsfile, which defines stages such as Build, Test, and Deploy. Each stage performs a specific task in sequence. The experiment also introduces Scripted Pipeline for understanding flexibility in pipeline creation.

Theory

Jenkins Pipeline

A Jenkins Pipeline is a suite of plugins that supports the implementation and integration of continuous delivery pipelines into Jenkins. It allows developers to define the entire build, test, and deployment process as code. Pipelines are defined in a file called Jenkinsfile, which is stored in the project repository. This approach is known as Pipeline as Code, which improves automation and version control of build processes.Pipelines consist of multiple stages, and each stage contains steps that perform tasks such as compiling code, testing applications, and deploying software.

Types of Jenkins Pipelines

Jenkins supports two types of pipelines:

1. Declarative Pipeline

2. Scripted Pipeline

1. Declarative Pipeline

Declarative Pipeline provides a simple and structured way of writing Jenkins pipelines. It use a predefined syntax that is easy to understand and maintain.

Features of Declarative Pipeline:

- Simplified syntax
- Easier to read
- maintain Supports predefined pipeline structure
- Suitable for most CI/CD workflows

Example structure:

```
pipeline {

agent any

stages {

stage('Build') {

steps {

// build steps

}

}}}
```

2. Scripted Pipeline

Scripted Pipeline is a more advanced and flexible way of writing pipelines using Groovy scripting language.

- Features of Scripted Pipeline:

- Highly flexible
- Uses Groovy programming syntax
- Suitable for complex pipeline logic
- Requires more programming knowledge

Example structure:

node {

stage('Build') {

// build steps

}

}

Jenkinsfile

A Jenkinsfile is a text file that contains the pipeline definition and is stored in the source code repository. Jenkins reads the Jenkinsfile and automatically executes the pipeline stages. Benefits of Jenkinsfile:

Pipeline configuration stored as code

Version control of build process

Easy collaboration among developers

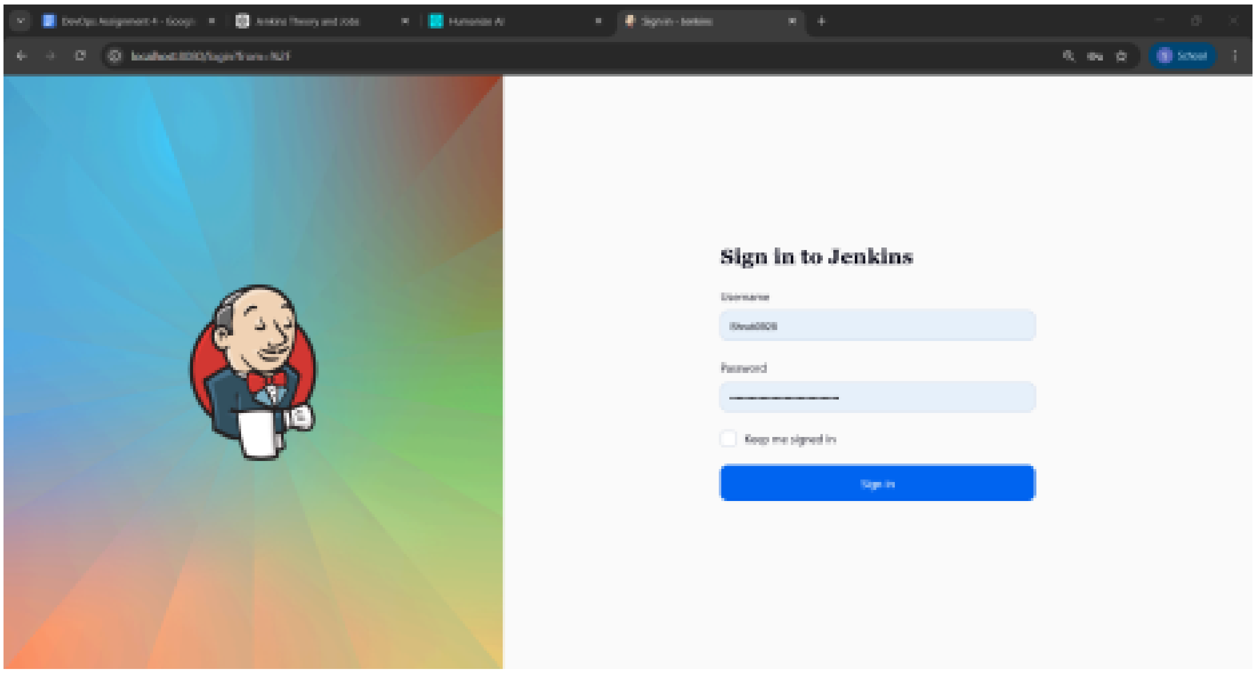
Automated CI/CD workflows.

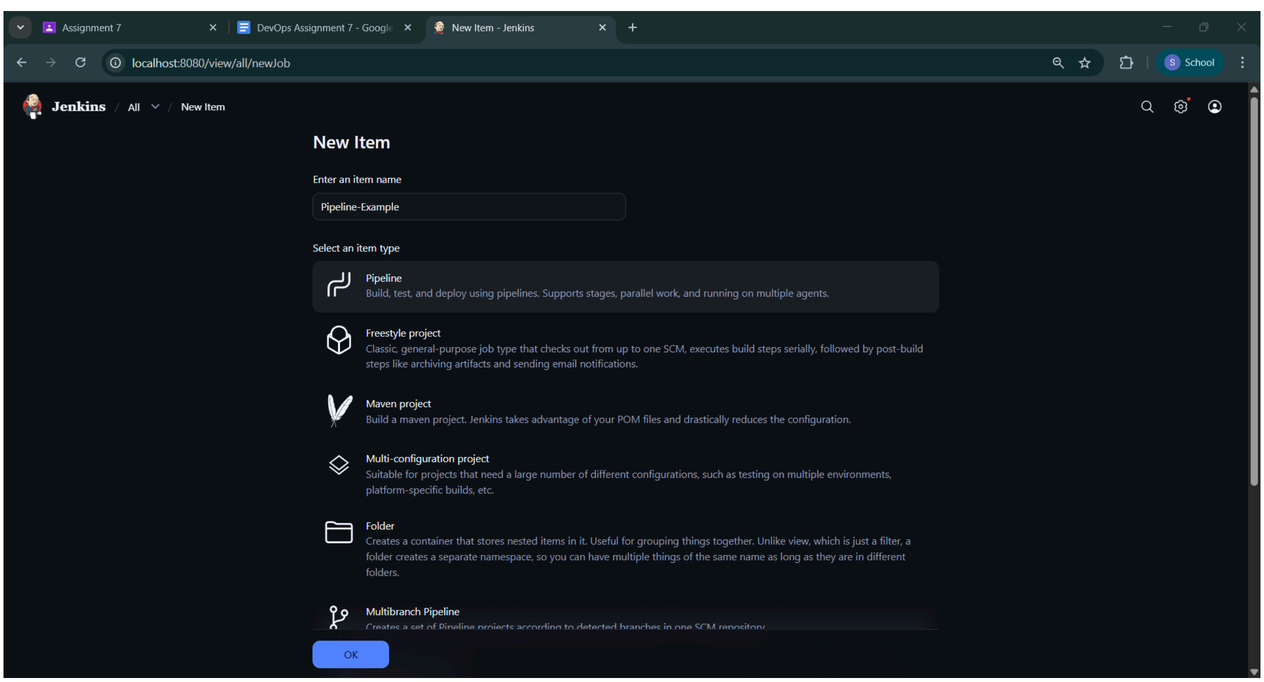
Description

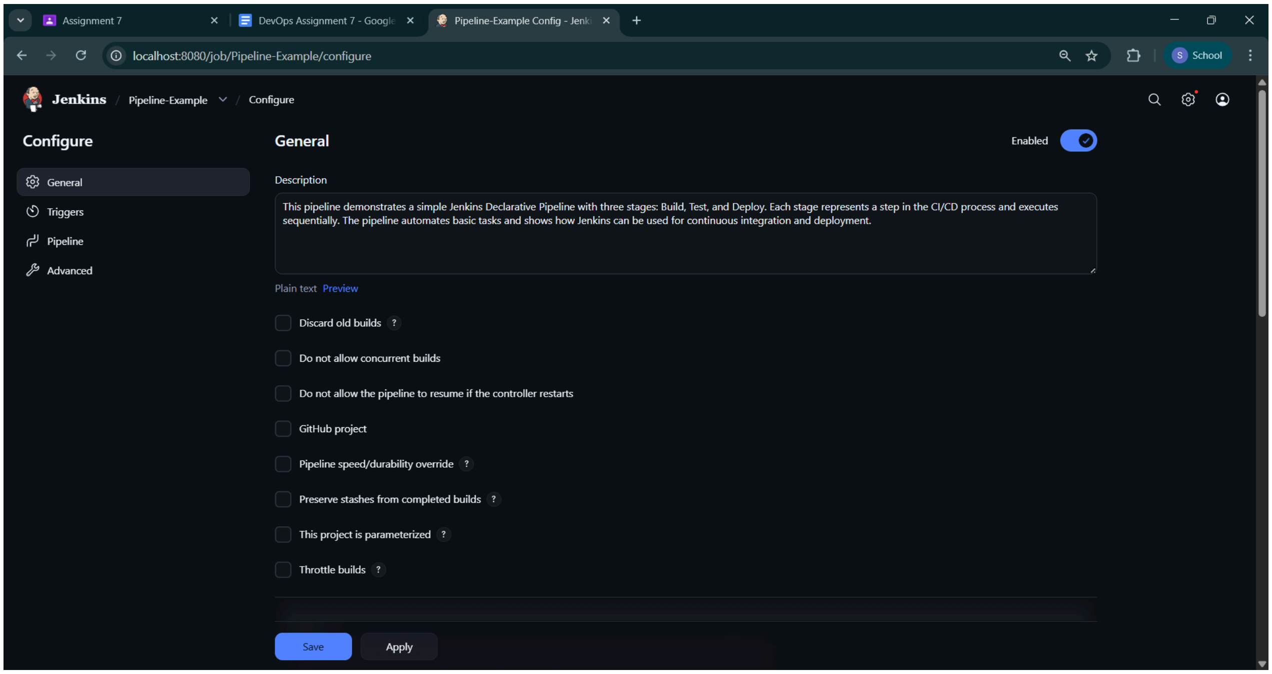
This experiment demonstrates the use of Jenkins Pipeline to automate the software development process. A Declarative Pipeline is created using a Jenkinsfile, which defines stages such as Build, Test, and Deploy. Each stage performs a specific task in sequence. The experiment also introduces Scripted Pipeline for understanding flexibility in pipeline creation.

Procedure

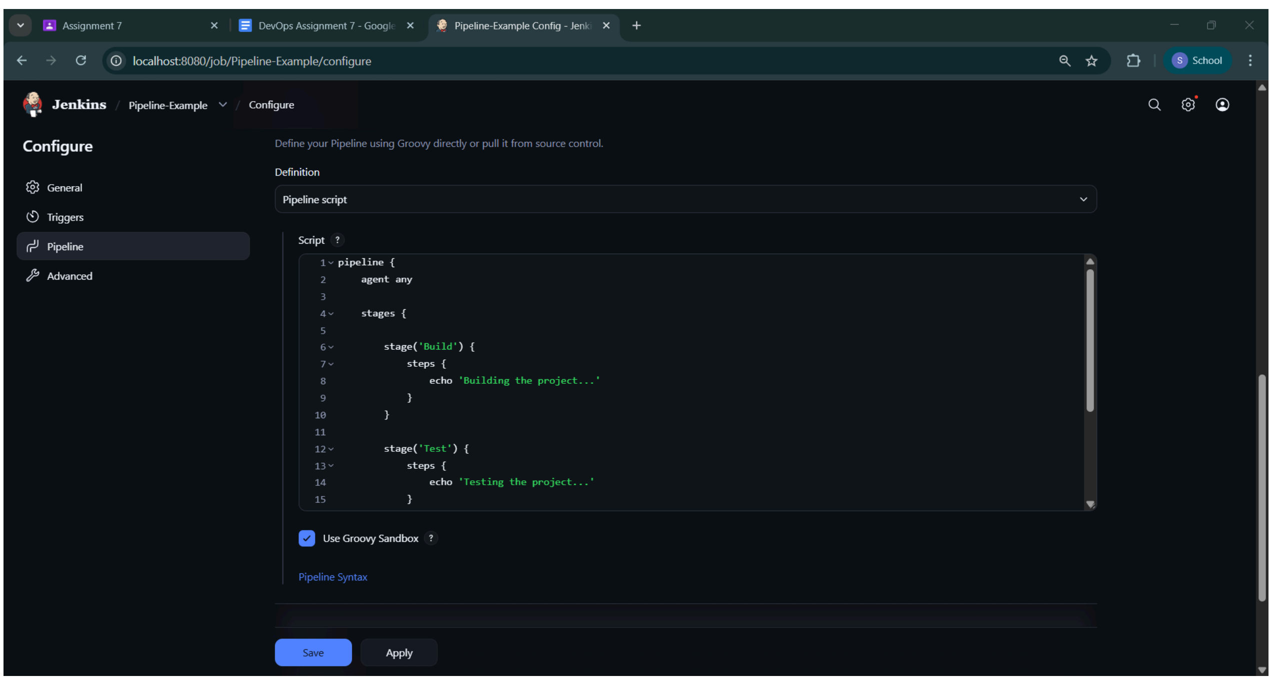
Step 1: Open Jenkins Dashboard (http://localhost:8080)





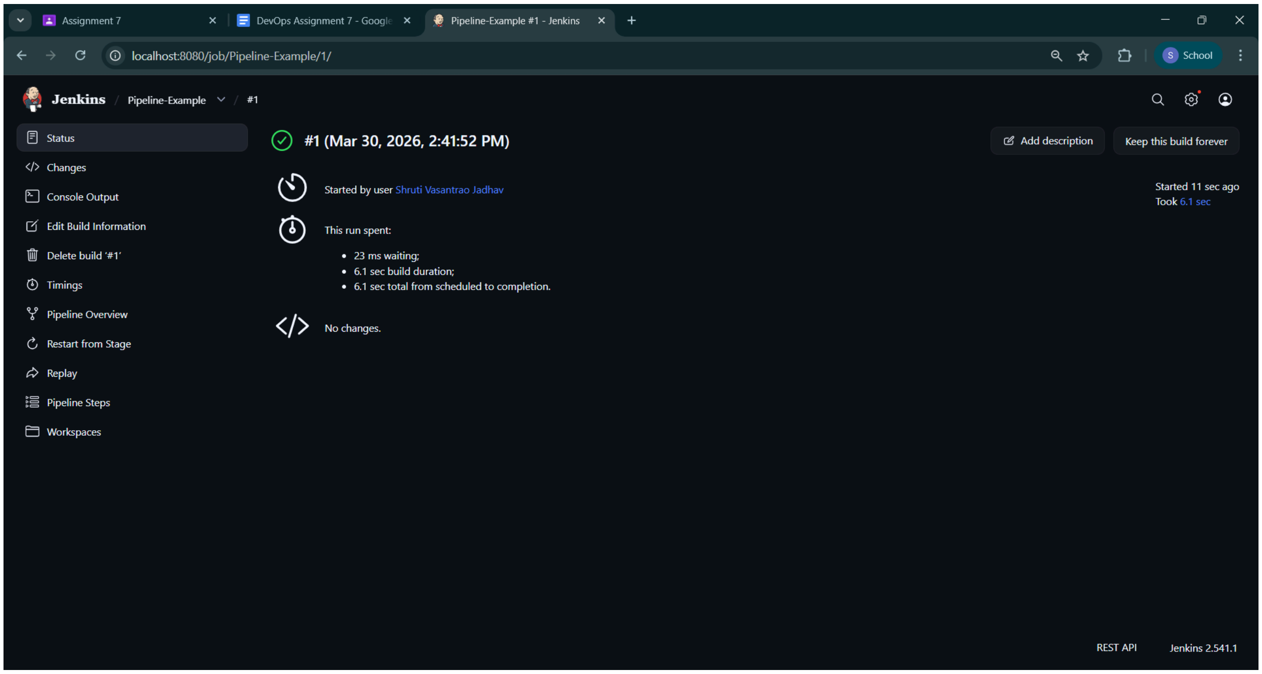


Assignment 7
DevOps Assignment 7 - Google
Pipeline-Example Config - Jenki
localhost:8080/job/Pipeline-Example/configure
School
Jenkins
Pipeline-Example
Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
This pipeline demonstrates a simple Jenkins Declarative Pipeline with three stages: Build, Test, and Deploy. Each stage represents a step in the CI/CD process and executes sequentially. The pipeline automates basic tasks and shows how Jenkins can be used for continuous integration and deployment.
Plain text Preview
Discard old builds
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Throttle builds
Save
Apply

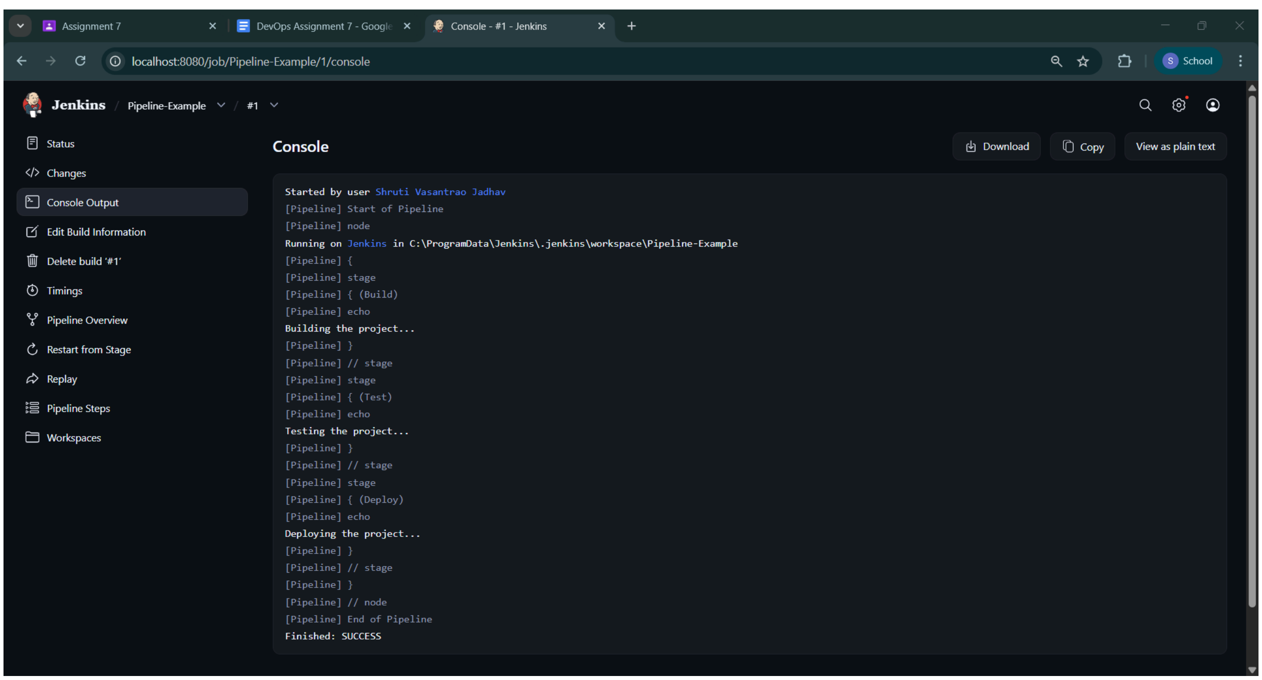


Assignment 7
DevOps Assignment 7 - Google
Pipeline-Example Config - Jenki
localhost:8080/job/Pipeline-Example/configure
School
Jenkins
Pipeline-Example
Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the project...'
}
}
stage('Test') {
steps {
echo 'Testing the project...'
}
Use Groovy Sandbox
Pipeline Syntax
Save
Apply

Step 2: Configure the Job (Write steps with screenshot)



Step 3: View Console Output (Screenshot)



FAQ:

1. What is the difference between stages and steps?

Feature	Stages	Steps
Definition	Stages are the main phases of a Jenkins pipeline such as Build, Test, and Deploy.	Steps are the individual tasks or commands executed inside a stage.
Purpose	Used to organize the pipeline into logical sections.	Used to perform actual operations like running scripts or commands.
Level	High-level components of a pipeline.	Low-level components inside stages.
Structure	A pipeline contains multiple stages.	Each stage contains one or more steps.
Example	Build, Test, Deploy	echo, sh, bat commands
Function	Helps in visualizing pipeline flow and progress.	Executes the actual work in the pipeline.

2. What is a Jenkinsfile?

A Jenkinsfile is a text file that contains the definition of a Jenkins Pipeline. It is written using either Declarative or Scripted syntax and is usually stored in a version control system like GitHub.

The Jenkinsfile defines all the stages, steps, and instructions required to build, test, and deploy an application. It helps automate the CI/CD process and ensures that the pipeline can be easily shared, reused, and maintained.

3. What is a Scripted Pipeline?

A Scripted Pipeline is a type of Jenkins Pipeline written using Groovy scripting language. It provides greater flexibility and control compared to Declarative Pipeline, allowing developers to write complex logic, use loops, conditions, and custom functions.

Unlike Declarative Pipeline, Scripted Pipeline does not follow a strict structure, making it more powerful but slightly harder to understand and maintain. It is mainly used in advanced scenarios where more customization is required.

Conclusion

In this experiment, the concepts of declarative and scripted pipelines were studied. A declarative Jenkins pipeline was created using a Jenkinsfile and executed successfully. This experiment helped in understanding how Jenkins pipelines automate build, testing, and deployment processes in CI/CD workflows.